# Deadlock Detection and Prevention in Business Process Models Using Petri Nets

Devansh Rai (IIB2024006), Geethika Gorakala (IIB2024024), Abhilasha Sharma (IIB2024501)
Business Process Management Project
Indian Institute of Information Technology Allahabad, Prayagraj, India

***Abstract*—Business process models drawn in BPMN can contain synchronisation errors, missing exception paths and conflicting resource use that leave a running case stuck before it reaches its end. Such deadlocks are hard to see by looking at a diagram. We present a six-step framework and a Python tool that (1) reads a BPMN 2.0 model, (2) maps it to a workflow Petri net, (3) detects deadlocks with a fast structural pattern check and an exhaustive state-space analysis of soundness and safeness, (4) explains the cause with a witness trace and siphon analysis, (5) applies a quick fix or a siphon-control place, and (6) re-verifies the corrected model for soundness. We apply it to a Loan Approval process built up in six models of increasing complexity, plus five models from other domains. Of the 14 models, 7 are unsound: four deadlock through control-flow errors, two through circular waits on shared resources, and one completes improperly. Every one of them is corrected and re-verified as sound, and every analysis finishes in under 2 ms. Our soundness verdicts match the independent pm4py/WOFLAN checker on all 14 models, the siphon-control places keep every good state (maximally permissive), and on the parallel-branch benchmark of Kräuter et al. our state spaces match theirs to within one state. A timed simulation of the final model with branch probabilities completes 100% of 1000 cases with a mean cycle time of 62.4 minutes.**

***Index Terms*—BPMN, Petri nets, deadlock, soundness, safeness, siphon, deadlock prevention, quick fixes, business process verification**

## I. Introduction

Organisations describe their workflows, such as loan approval, order handling and claim processing, as business process models, most often in the Business Process Model and Notation (BPMN) [6]. BPMN is easy to read but lacks precise formal semantics, so a model can look correct and still contain behavioural errors [1], [2]. The most harmful of these is a *deadlock*: a case reaches a state in which no activity can continue, although the process has not finished. In a loan process this means an application that is never approved or rejected.

Petri nets give a precise, graphical and mathematical model of states, activities and concurrency [5]. Once a BPMN model is mapped to a Petri net, deadlocks become reachable markings in which no transition is enabled, and they can be found mechanically [1].

**Research question.** *How can Petri nets be used to detect and prevent deadlocks in business process models while preserving correct workflow behaviour?*

**Contributions.** (i) A six-step framework that links BPMN modelling, Petri-net conversion, detection, diagnosis, correction and re-verification; (ii) a Python implementation with a Streamlit interface that combines the soundness and safeness checking and quick fixes of Kräuter et al. [1], the BPMN-to-Petri-net mapping, mapping check and evolution operations of Zhang et al. [2], and the siphon-based deadlock prevention of Wang et al. [3], extended with shared resources, time and probabilities; (iii) a Loan Approval case study of six models, from a simple sequence to a timed model with shared resources, with before/after results and a scalability benchmark.

## II. Related Work

**Soundness checking of BPMN.** Kräuter et al. [1] present a BPMN soundness checker that is *instantaneous* (under 500 ms), *comprehensible* and *fixable*. It explores the full state space with a breadth-first search and checks option to complete, proper completion, no dead activities and *safeness* (no sequence flow ever holds two tokens) on the fly. A task is encoded as a single step, which keeps the state space small. Violations are highlighted in the diagram and counterexamples are replayed as moving tokens. For common violations the tool offers IDE-style *quick fixes*. A split/join mismatch, for example, can be repaired either at the join or at the split, and the user chooses. Their tool does not check custom properties, data or time, and it has no notion of shared resources.

**Formal mapping and model evolution.** Zhang et al. [2] map BPMN to hierarchical coloured Petri nets (HCPN) with element-level rules (start, intermediate and end events, tasks, parallel and exclusive gateways, sub-processes, sequence and message flows). They check the mapping with a correspondence and connectivity test (their Algorithm 1). They describe changes to a process as atomic *insert*, *delete* and *replace* operations found by comparing two versions of the model (Algorithm 2), and they verify each evolved model by state-space analysis in CPN Tools (dead markings, dead transitions, reachability).

**Siphon-based deadlock prevention.** Wang et al. [3] prevent deadlocks in Petri nets in which every resource place initially holds one token. In a first stage, mixed-integer programming finds a specific kind of emptiable siphon, avoiding both full siphon enumeration and reachability analysis. In a second stage, a monitor (control place) with unit arc weights is added for each such siphon. The resulting supervisor is *maximally permissive*: it forbids only the states that lead to a deadlock.

TABLE I
PETRI NET ELEMENTS AND THEIR MEANING IN A PROCESS

| Element | Meaning in a business process |
|---|---|
| Place (◯) | a condition or state, e.g. “credit checked” |
| Transition (□) | an activity or routing event |
| Token (●) | where the case currently is |
| Arc (→) | connects places and transitions |

Our work combines these three lines for one practical goal: finding, explaining and removing deadlocks in BPMN models that also share resources. The correctness criterion is the soundness of workflow nets [4].

## III. PRELIMINARIES

**Petri net.** A Petri net is $N = (P, T, F, W)$ with finite disjoint sets of places $P$ and transitions $T$, arcs $F \subseteq (P \times T) \cup (T \times P)$ and weights $W : F \to \mathbb{N}^+$. A *marking* $M : P \to \mathbb{N}$ gives the number of tokens in each place. Table I lists the elements.

**Firing rule.** Transition $t$ is enabled in $M$, written $M[t\rangle$, if $M(p) \geq W(p,t)$ for every input place $p \in {}^\bullet t$. Firing $t$ gives $M'(p) = M(p) - W(p,t) + W(t,p)$.

**Workflow net.** A net with one source place $i$ and one sink place $o$, initial marking $M_0 = [i]$ and final marking $M_f = [o]$. Shared resources add places whose free units are present in both $M_0$ and $M_f$.

**Deadlock.** A reachable marking $M \in R(N, M_0)$ is *dead* if no transition is enabled. We call it a *deadlock* if, in addition, $M(o) = 0$, and an *improper completion* if $M(o) \geq 1$ but $M \neq M_f$:

$$M \in R(N, M_0), \quad M \neq M_f, \quad \nexists\, t \in T : M[t\rangle. \qquad (1)$$

**Soundness** [4]. A workflow net is sound if (1) from every reachable marking $M_f$ can still be reached (option to complete), (2) whenever $o$ is marked, no other token remains (proper completion), and (3) every transition can fire in some run (no dead transitions). Following [1] we also check *safeness*: no place that stands for a sequence flow ever holds more than one token. An unsafe flow points to a missing synchronisation.

**Siphon** [3], [5]. A non-empty set $S \subseteq P$ is a siphon if ${}^\bullet S \subseteq S^\bullet$, that is, every transition that puts a token into $S$ also takes one from $S$. A siphon that becomes empty can never be marked again, so every transition that needs one of its places is dead from then on.

## IV. PROPOSED METHODOLOGY

Fig. 1 shows the six steps. A corrected model goes back through the whole pipeline, and it is accepted only when it is sound.

### A. Step 1: Process modelling

Each model is drawn in BPMN 2.0 (Camunda Modeler or bpmn.io) and saved as XML. Extra information is written in the standard `documentation` field, so the files stay valid BPMN. We use the keys `resources: officer=1`, `acquire/release/uses: officer`, `time: 20` and `prob: 0.3`.

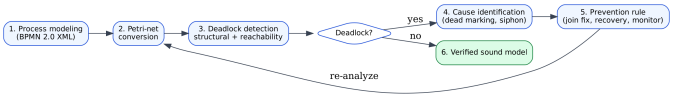



Fig. 1. Petri-net-based deadlock detection and prevention framework.

TABLE II
BPMN TO PETRI NET MAPPING

| BPMN element | Petri net pattern |
|---|---|
| Sequence flow | one place |
| Start event | source place $i$ (1 token) and a transition |
| End event | transition into sink place $o$ |
| Task | one transition (input place → output place) |
| AND-split / AND-join | one transition with several output / input arcs |
| XOR-split / XOR-join | one transition per outgoing / incoming branch |
| OR-split | one transition per non-empty subset of branches |
| Shared resource | a place holding its free units; *acquire* adds an input arc, *release* an output arc |
| Element without outgoing flow | a “stuck” place that is never consumed |

### B. Step 2: BPMN to Petri net

Table II gives the mapping. It follows the element rules of Zhang et al. [2] without colours and hierarchy, and like the pragmatic encoding of [1] it fires a task as one transition instead of separate start and end events. Every sequence flow becomes a place, so every token can be traced back to an arrow in the diagram. After the mapping, the correspondence and connectivity check of [2] (Algorithm 1) confirms that every BPMN element has a transition, every sequence flow has a place, and that place is produced by the flow’s source and consumed by its target.

### C. Step 3: Deadlock detection

*a) Structural check.* Kräuter et al. [1] list the typical causes of each soundness violation: a parallel gateway that waits for flows that never all arrive (option to complete), an exclusive gateway that merges parallel flows (safeness and proper completion), and a disconnected activity (no dead activities). They find the cause after a counterexample has been found. We also search for these causes directly in the BPMN graph, before any state is explored, as six patterns: P1, an XOR/OR-split whose branches are closed by an AND-join; P2, an AND-join at a loop entry; P3, an AND-split closed by an XOR-join; P4, an element with no outgoing flow; P5, an unreachable element; and P6, two resources taken in opposite orders on different paths (a circular wait, see Step 4). For P1 we walk forward from every branch of a split. The pattern holds if one branch reaches an incoming flow $a$ of the join but not another incoming flow $b$, while a second branch reaches $b$ but not $a$.

*b) State-space analysis.* As in [1], a breadth-first search builds the reachability graph from $M_0$, storing for each new

marking the marking and transition it was reached from. Every dead marking other than $M_f$ is reported together with its *witness trace*, the firing sequence that leads to it. A reverse search from $M_f$ checks the option to complete, and the three soundness rules and safeness are evaluated. The search stops at 20,000 states, or when a place holds more than 50 tokens (treated as unbounded). In that case the verdict is reported as *inconclusive* rather than unsound, and the states at the cut-off are not counted as dead.

### D. Step 4: Cause identification

Like [1], we find the cause from the last state of the counterexample. For each dead marking we list the marked places and, for each one, the transition that is waiting and the input places it still lacks. Those transitions are mapped back to their BPMN elements and outlined in red in the diagram. For resource deadlocks we look for emptiable siphons, as in [3]. Wang et al. find them with mixed-integer programming. Our nets are small, so we enumerate the minimal siphons (leaving out the source place, which empties by design) and keep those that are marked in $M_0$ but empty in a dead marking. Such a siphon identifies the circular wait.

### E. Step 5: Prevention rules

Control-flow errors are repaired with *quick fixes* in the sense of [1], whose goal is matching gateways. A split/join mismatch has two candidate fixes, at the join or at the split. Both are generated and re-verified. The join fix is applied by default because it keeps the business decision made at the split.

- *P1/P2:* replace the AND-join with an XOR-join (alternative for P1: make the split an AND-split).
- *P3:* replace the XOR-join with an AND-join (alternative: make the split an XOR-split).
- *P4 (missing recovery path):* redesign the model with a resubmission loop and a coordinated rejection path, so the join always receives a result.
- *P6 (circular wait):* following the siphon-control stage of [3], for each problem siphon $S$ add a control place $V_S$, built from a place invariant [5]. With $d(t) = \sum_{p \in S} \big(W(t,p) - W(p,t)\big)$, we add an arc $t \to V_S$ of weight $d(t)$ when $d(t) > 0$ and an arc $V_S \to t$ of weight $-d(t)$ when $d(t) < 0$, and set $M_0(V_S) = M_0(S) - 1$. Then $M(V_S) - M(S) = -1$ holds in every reachable marking, so $M(S) \geq 1$ and $S$ is never emptied. All our resources have one unit, the net class of [3], and all monitor arcs then have weight 1. To check permissiveness we project the states of the controlled net onto the original places and count how many *good* states (from which $M_f$ is still reachable) are kept. A controller that keeps all of them is maximally permissive.

The gateway rules are applied to the BPMN XML itself, so the corrected model can be opened again in the modeller.

### F. Step 6: Re-analysis and verification

Following the evolution view of [2], a correction is a change of the process model made of atomic *insert*, *delete* and *replace* operations. We compute these operations between the faulty and the corrected model (their Algorithm 2, with operations labelled as in their Section 4.4), so the user sees exactly which part was changed. Steps 2–3 are then repeated on the corrected model. It is accepted when it has no dead marking other than $M_f$, it is sound, and every path that completed correctly before still completes. None of the three approaches handles time, and [2] names timed nets as future work. As our own extension, we run a timed simulation of the final model. Transitions take their inputs when they start and give their outputs after their duration, XOR branches are drawn with the given probabilities, and we report the completion rate and cycle times.

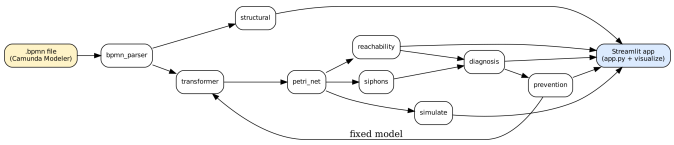



Fig. 2. Module structure of the tool.

TABLE III
MODULES AND THEIR MAIN FUNCTIONS

| Module | What it does |
|---|---|
| `bpmn_parser` | `parse_bpmn()` reads the XML into nodes, flows, annotations |
| `transformer` | `bpmn_to_petri()` applies Table II; `check_mapping()` |
| `petri_net` | `enabled()`, `fire()` implement the firing rule |
| `structural` | `check()` finds patterns P1–P6 |
| `reachability` | `analyse()` builds the graph, finds dead markings, checks soundness and safeness |
| `siphons` | `minimal_siphons()`, `problem_siphons()` |
| `diagnosis` | `explain()` says which transition waits for which place |
| `prevention` | `fix_gateways()`, `alternative_fixes()`, `add_monitors()`, `permissiveness()` |
| `evolution` | `detect_changes()` lists insert/delete/replace operations |
| `simulate` | `run_cases()` runs the timed, probabilistic simulation |
| `app.py` | Streamlit interface: diagram, net, detection, fix, token game, simulation |

## V. IMPLEMENTATION

The tool is written in Python 3 with about 1,400 lines of code. Fig. 2 shows how the modules fit together. We implemented the core analysis ourselves so that every step can be explained. The library pm4py [7] is used only to cross-check our soundness results.

**Input (dataset).** The dataset is a set of 14 BPMN 2.0 XML process models: nine Loan Approval models (M1–M6 and three fixed versions) and five demonstration models from other domains. The tool also accepts a Petri net as JSON (places, transitions, arcs, initial and final marking).

**Output.** The Petri net and reachability graph with dead states in red; dead markings with witness traces; the cause report and siphons; the fixed BPMN file; a before/after table;

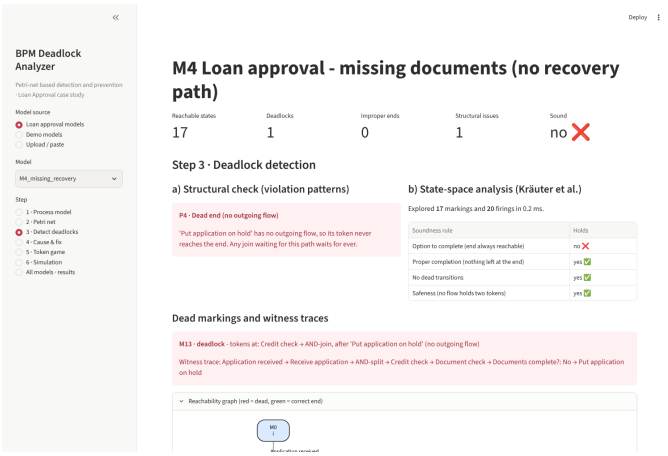



Fig. 3. Detection step in the Streamlit application for model M4.

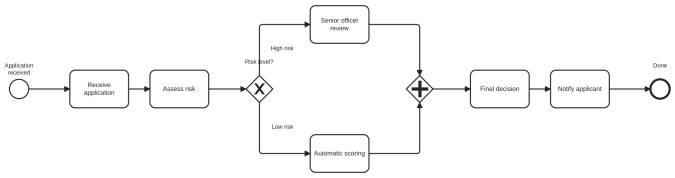



Fig. 4. M3: the XOR-split "Risk level?" is closed by an AND-join.

and simulation statistics. The interface (Fig. 3) also offers a token game in which the user fires enabled transitions by hand.

## VI. Case Study and Results

### A. Loan Approval models

- **M1** is a sequence: receive application, check eligibility, make decision, notify.
- **M2** runs the credit check and document check in parallel (AND-split/AND-join).
- **M3** routes by risk level with an XOR-split, but closes the branches with an AND-join (Fig. 4).
- **M4** lets the document check fail, but the "put on hold" path has no outgoing flow (the scenario of our proposal).
- **M5** runs a credit assessment and a fraud check in parallel. Both need the single loan officer and the single credit-system session, and they take them in opposite orders.
- **M6** combines all the corrections and adds durations, branch probabilities and a shared officer (Fig. 5).

### B. Detection and correction results

Table IV summarises all 14 models.

**M3.** Both dead markings hold a single token in front of the AND-join. For example, the trace *Receive application* → *Assess risk* → *Risk level?: High* → *Senior officer review* leaves the join waiting for the "Automatic scoring" branch, which was never started. Pattern P1 points at the same pair of gateways. Replacing the join with an XOR-join gives a sound net with 12 states.

**M4.** The dead marking has tokens in "Credit check → AND-join" and in the stuck place after "Put application on hold". The approval waits for a "documents complete" token that can never come (Fig. 6). No gateway change can fix this, so the model is redesigned as M4-fixed. Missing documents are requested and the check is repeated, and if they do not arrive the documents are marked as failed. Either way the AND-join receives a result, and a decision gateway then approves or rejects the loan. M4-fixed is sound with 32 states.

**M5.** The witness trace *AND-split* → *Assign loan officer* → *Open credit system (fraud)* reaches a dead marking in which each branch holds one resource and waits for the other. The analysis finds the siphon $S = \{r_{officer}, r_{credit}, p_5, p_8\}$, where $p_5$ and $p_8$ are the places in which a branch holds both resources. $S$ holds 2 tokens in $M_0$ and none in the deadlock. The control place $V_1$, with 1 token, is taken by the first resource request of each branch and returned when the branch releases both resources (Fig. 7). So only one branch can be inside its critical section at a time, and the net becomes sound with 19 states. The alternative BPMN-level fix (M5-fixed: both branches take the officer first) is also sound. Pattern P6 detects the opposite ordering directly in the diagram.

**Demo models.** D2 and D5 show P1 and P2 in other domains. D3 is unsound without a deadlock: the XOR-join lets both parallel branches through, so "Update team calendar" runs twice and two tokens reach the end. This improper completion is caught by soundness rule 2, and D3 is the only model that is not safe: both flows after the XOR-join can hold two tokens. D4 repeats the circular-wait pattern with a bed and a doctor.

**Alternative quick fixes.** The split-side fixes of [1] are also sound: making "Risk level?" in M3 an AND-split (12 states), making "Claim size?" in D2 an AND-split (10 states), and making the AND-split of D3 an XOR-split (10 states). They change what the business does (e.g. every application gets both reviews), so they are offered but not applied by default.

**Permissiveness of the control places.** In M5 the original net has 19 good states and the controlled net keeps all 19; in D4 it keeps 18 of 18. Both controllers are therefore maximally permissive in the sense of [3], and all their arcs have weight 1.

**Evolution operations.** The change detection of [2] describes each hand correction: M3→M3-fixed is a single *replace* (the join's type), M5→M5-fixed is three *replace* operations (the fraud branch's flows re-ordered), and the M4 redesign consists of 20 *insert*, 1 *delete* and 7 *replace* operations. The mapping check of [2] reports no error for any of the 14 models.

**Cross-check.** pm4py's WOFLAN check [7] reads the control flow only. On the control flow of all 14 models, its soundness verdict matches ours. All our analyses take under 2 ms per model.

### C. Scalability

To compare with [1], we generated their worst-case benchmark: an AND-split into $n$ branches of $m$ tasks each. Table V shows that our state counts are theirs plus one in every case (our start event has its own transition), so the two encodings are equally compact. Our Python tool is slower than their Rust

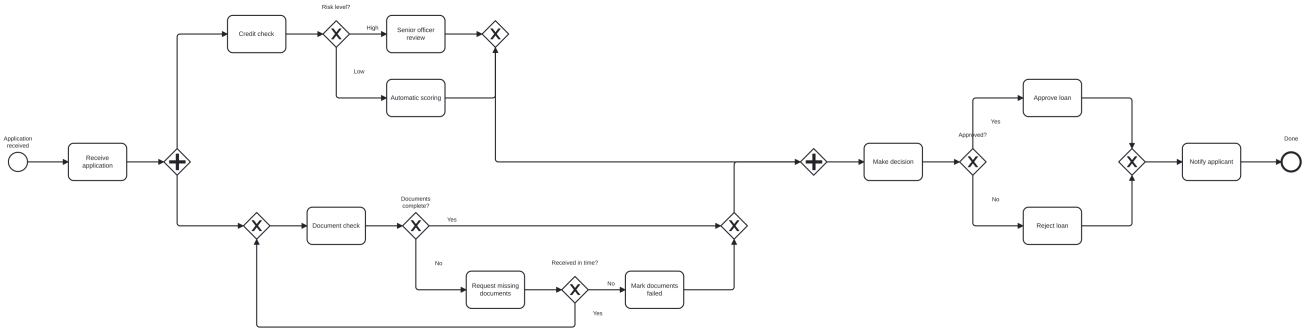



Fig. 5. M6: final corrected Loan Approval model with resubmission loop, coordinated rejection and risk routing.

TABLE IV
ANALYSIS RESULTS BEFORE AND AFTER CORRECTION (S = REACHABLE STATES, D = DEADLOCKS, I = IMPROPER COMPLETIONS)

| Model | $\|P\|$ | $\|T\|$ | S | D | I | Sound | Pattern | Correction | Sound after | S after |
|---|---|---|---|---|---|---|---|---|---|---|
| M1 sequential | 7 | 6 | 7 | 0 | 0 | yes | – | – | – | – |
| M2 parallel | 11 | 9 | 11 | 0 | 0 | yes | – | – | – | – |
| M3 wrong join | 12 | 11 | 8 | 2 | 0 | no | P1 | AND-join → XOR-join | yes | 12 |
| M4 missing recovery | 14 | 12 | 17 | 1 | 0 | no | P4 | redesign (M4-fixed) | yes | 32 |
| M5 resource deadlock | 17 | 13 | 20 | 1 | 0 | no | P6 | control place $V_1$ / fixed order | yes | 19 / 19 |
| M6 full timed | 30 | 31 | 82 | 0 | 0 | yes | – | – | – | – |
| D1 order fulfilment | 13 | 12 | 13 | 0 | 0 | yes | – | – | – | – |
| D2 insurance claim | 10 | 9 | 7 | 2 | 0 | no | P1 | AND-join → XOR-join | yes | 10 |
| D3 leave request | 10 | 9 | 25 | 0 | 1 | no | P3 | XOR-join → AND-join | yes | 10 |
| D4 hospital admission | 16 | 12 | 19 | 1 | 0 | no | P6 | control place $V_1$ | yes | 18 |
| D5 purchase order | 10 | 9 | 3 | 1 | 0 | no | P2 | AND-join → XOR-join | yes | 10 |

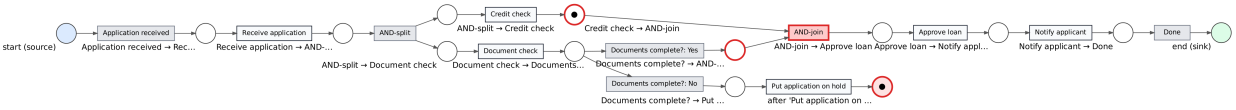



Fig. 6. Petri net of M4 in its dead marking. Red: the waiting AND-join, the empty place it needs, and the stuck token.

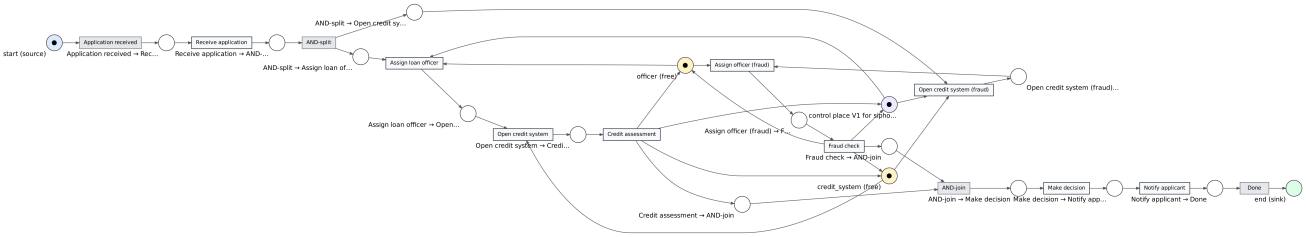



Fig. 7. M5 with the siphon control place $V_1$ (purple) added by the prevention step.

tool but still checks all rows up to 9,265 states in about 0.35 s. At 15 parallel branches it reaches the 20,000-state limit and reports an inconclusive verdict.

### D. Simulation of the final model

With the durations and probabilities shown in the model (e.g. 30% high-risk applications, 25% incomplete documents, 80% of requested documents received), 1000 simulated cases of M6 all complete. The mean cycle time is 62.4 minutes, the median 50 and the 90th percentile 78 (Fig. 8). In the uncorrected M5, when both branches start at the same moment, every simulated case deadlocks. With the control place, every case completes.

## VII. DISCUSSION

The two detection methods complement each other. The structural check is instant and names the faulty gateways, but it is a heuristic: it can miss errors and, in unusual structures, raise false alarms. The state-space analysis is complete and gives a concrete witness trace, but the number of states can grow exponentially with concurrency (state explosion), as the benchmark shows. Our largest case-study model has 82 states, far below the limit. For much larger models, partial-order

TABLE V
PARALLEL-BRANCH BENCHMARK OF [1] (STATES)

| Branches | Length | Ours | [1] | Our verdict |
|---|---|---|---|---|
| 5 | 1 | 36 | 35 | sound |
| 10 | 1 | 1,028 | 1,027 | sound |
| 5 | 3 | 1,028 | 1,027 | sound |
| 5 | 5 | 7,780 | 7,779 | sound |
| 3 | 10 | 1,335 | 1,334 | sound |
| 3 | 20 | 9,265 | 9,264 | sound |
| 15 | 1 | $\geq$20,000 | 32,771 | inconclusive |

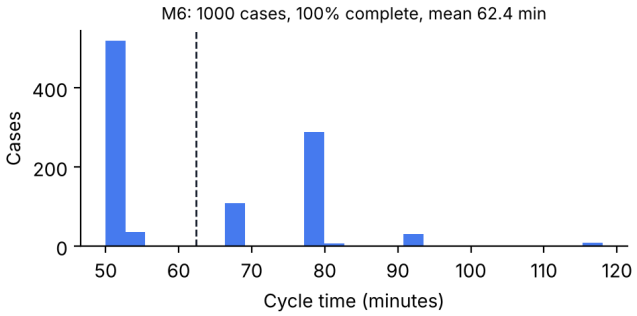



Fig. 8. Cycle-time distribution of 1000 simulated M6 cases.

reduction [1], hierarchical decomposition into modules [2], or finding problem siphons by mixed-integer programming instead of from the state space [3] would be needed.

The corrections preserve correct behaviour. A gateway fix only removes runs that ended in a deadlock, and every run that completed before still completes. The siphon control place only delays one branch until the other releases its resources, so no activity is removed, and the permissiveness check confirms that no good state is lost.

**Limitations.** OR-joins are approximated as XOR-joins. Sub-processes, pools, boundary events and message flows are not translated (the hierarchical mapping of [2] covers sub-processes and message flows). Each model is analysed for a single case, so resource conflicts between concurrent cases are not covered. Problem siphons are found from the state space, not by the MIP of [3]. The times and probabilities in M6 are assumed values, not data from a real bank. The recovery redesign for P4 cannot be automated, because it is a business decision.

## VIII. CONCLUSION AND FUTURE WORK

We showed that Petri nets give a practical way to detect, explain and prevent deadlocks in BPMN models. Combining pattern-based structural checks with state-space and siphon analysis found every injected error in our 14 models, identified its cause, and confirmed each correction as sound, with control places that are maximally permissive. Future work follows the three papers we build on: MIP-based siphon detection [3] for large nets; coloured tokens for several concurrent cases and hierarchical sub-processes [2], so that resource conflicts between loan applications can be analysed; and replaying counterexamples on the BPMN diagram itself [1]. Further items are full OR-join semantics and learning times and probabilities from event logs.

## REFERENCES


[1] T. Kräuter, P. Stünkel, A. Rutle, H. König, and Y. Lamo, "Instantaneous, comprehensible, and fixable soundness checking of realistic BPMN models," arXiv:2407.03965 [cs.SE], 2024.

[2] Z. Zhang, F. Ni, J. Liu, N. Chen, and X. Zhou, "Collaborative modeling of BPMN and HCPN: Formal mapping and iterative evolution of process models for scenario changes," *Information*, vol. 16, no. 4, Art. no. 323, 2025, doi: 10.3390/info16040323.

[3] S. G. Wang, X. Guo, O. Karoui, M. C. Zhou, D. You, and A. Abusorrah, "A refined siphon-based deadlock prevention policy for a class of Petri nets," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 53, no. 1, pp. 191–203, 2023, doi: 10.1109/TSMC.2022.3174421.

[4] W. M. P. van der Aalst, "The application of Petri nets to workflow management," *J. Circuits, Systems and Computers*, vol. 8, no. 1, pp. 21–66, 1998.

[5] T. Murata, "Petri nets: Properties, analysis and applications," *Proc. IEEE*, vol. 77, no. 4, pp. 541–580, 1989.

[6] Object Management Group, *Business Process Model and Notation (BPMN), Version 2.0*, 2011.

[7] A. Berti, S. J. van Zelst, and W. M. P. van der Aalst, "Process Mining for Python (PM4Py): Bridging the gap between process- and data science," in *Proc. ICPM Demo Track*, 2019, pp. 13–16.